

BharatTruck — MVP Product Requirements Document (PRD)

Product	BharatTruck (built on the LogisticOS microservices platform)
Document	MVP PRD v1.0
Status	Draft for review
Owner	Founder / Eng
Created	2026-06-30
Target ship	31 August 2026 (≈ 8-9 weeks, hard deadline)
North-Star metric	Completed Paid Trips
Goal of this MVP	Prove feasibility — that a fleet owner / driver can run a real interstate freight trip end-to-end through the platform instead of phone+WhatsApp. Not a scale or compliance milestone.

How to read this doc: **Part 0** is the one-page overview (read this if nothing else). **Part 5** is the detailed scope of each module separately. **Part 7** is the standalone Data Requirements list (so data-dependent features can be procured separately). **Part 11** is the "what you actually hand over at MVP completion" checklist.

PART 0 — One-Page Overview

What BharatTruck is: A two-sided freight marketplace that replaces the manual "call a transport guy on WhatsApp" process with a self-serve flow. A **Shipper** posts a load; **Drivers / Fleet Owners** compete for it via auction or strike a direct contract; the truck is matched, the trip is GPS-tracked live, delivery is proved via an OTP + a tamper-evident photo ledger, and money settles through the platform. The product's promise is **transparency & trust for shippers** and **comfort & analytics for fleet owners**.

Who it's for (we optimize for the supply side):

- **Shipper** — an SME with full/part truckloads to move city-to-city. *Separate persona, separate app.*
- **Driver + Fleet Owner** — *one combined persona, one app.* Role is **truck-derived**: register 1 truck → you're a Driver; add a 2nd → you're a Fleet Owner; a fleet's truck can be assigned to any of its drivers and reflects in that driver's app. Roles are fluid and migratable.
- **Ops** — a single on-call operator running a web console; only touches the flow on exceptions.

The MVP delivers (one line each):

1. **Identity & KYC** — sign up, KYC-gated onboarding (Driver: PAN+Aadhaar; Truck: Vahan/RC; Shipper KYC above ₹50k), manually approved by ops via a console, "Verified" badge.
2. **Load + Booking** — shipper posts a point-to-point FTL/LTL load; **auction** (many drivers bid, shipper picks) **or direct contract** (1:1); capped 5-round time-boxed negotiation; booked price is final.
3. **Pricing** — a cost-breakdown engine (fuel / driver / per-km) as the price anchor, with the RL dynamic-pricing layer integrated (synthetic data at MVP — see Data Requirements).
4. **Payments** — multi-mode (escrow via Razorpay / direct / cash) with per-hour detention charges; *legal/compliance is explicitly deferred for the feasibility test.*
5. **Live Tracking** — driver app streams GPS; shipper sees the truck on a live Google Map; geofenced pickup/drop; >1hr halt alerts.
6. **POD + Cargo Ledger** — receiver-email **OTP closes the trip**; every checkpoint photo carries EXIF GPS, is stored in our data sink, and its **hash is written to a blockchain ledger clubbed to the order** = the legal record.
7. **Ops Console** — one web console with full visibility across all drivers/fleets/trips, KYC approval queue, and manual override for exceptions.

Delivery surfaces: 3 web/PWA apps now (Shipper, Driver/Fleet, Ops Console) for fast iteration; the **Driver app gets wrapped in Capacitor** → **native Android** for reliable background GPS before the real pilot.

Reality check: The platform is **~30% built — mostly scaffolding**. Payment, tracking, cargo-ledger, KYC, and the ops console are stubs or empty; both mobile apps currently fail to build. The 8-week plan (Part 10) is aggressive and assumes ruthless focus on the **single happy-path: one shipper → one driver → one completed, tracked, proven, paid interstate trip**.

Out of scope for MVP: multi-pickup/multi-drop, partial delivery, mid-trip price changes, ratings for drivers, intracity loads, WhatsApp/voice booking, real-time auto driver-matching, and passing full payment/legal compliance.

PART 1 — Executive Summary

1.1 The problem

Booking an intercity truck in India today runs on relationships and phone calls. A shipper who needs to move goods must *know* a transport broker / fleet owner, call around, haggle verbally, and then trust an opaque process for pickup, transit, and delivery. There is no transparency for the shipper (where is my truck? did it actually deliver?) and no tooling or analytics for the fleet owner (which loads, what price, which driver, how much idle time). The coordination is entirely manual.

1.2 The solution

BharatTruck digitizes this coordination as a self-serve marketplace:

- Shippers post loads and receive **competitive, transparent pricing** (auction or guided contract).
- Fleet owners and drivers get a **low-friction, vernacular-friendly app** to find loads, assign trucks, and run trips with built-in navigation hand-off and proof-of-delivery.
- Every trip is **GPS-tracked** and **cryptographically proven** (photo+OTP ledger), creating trust without a middleman.

1.3 What "MVP done" means

A real shipper posts a real interstate load. A real fleet owner's truck (KYC-verified, Vahan-checked) wins it via auction or direct contract. The driver runs the trip; the shipper watches it live on a map; the receiver confirms with an OTP; checkpoint photos are ledgered; money settles through the platform. **That single end-to-end "Completed Paid Trip" — done by a user who is not us — is the MVP.** Everything in this doc exists to make that loop work and repeatable.

1.4 Non-goals (MVP)

Scale, profitability, full legal/RBI payment compliance, polished design, automated dispatch, and breadth of edge-case handling are **explicitly not** MVP goals. Ops fills every gap by hand.

PART 2 — Users, Personas & Roles

2.1 Personas

Persona	App / Surface	Description	Friction tolerance
Shipper	Shipper PWA (separate)	SME with >1 truckload to move city-to-city, interstate or intrastate. Wants transparency & trust.	Medium — can handle a clean form.
Driver / Fleet Owner	Driver-Fleet PWA → Capacitor Android (combined)	Truck owner-operators and transporters. Primary audience — optimize for them. May be non-literate → icon-led, vernacular UX.	Very low — near-zero friction required.
Ops	Ops Console (web)	Single on-call operator. Approves KYC, watches trips, resolves exceptions, can override.	N/A (internal).
Receiver / Consignee	No account — email only	The party goods are delivered to; may differ from shipper. Receives & enters delivery OTP.	None — no app, no login.

2.2 The truck-derived role model (core design decision)

Role is **not** a fixed account type — it is **derived from trucks attached to the account**:

- Register **1 truck** → the user functions as a **Driver**.
- Add a **2nd truck** → the user becomes a **Fleet Owner** (manages multiple trucks + affiliated drivers).
- A single person can be **both** a Driver and a Fleet Owner simultaneously (drives one truck, owns another with a hired driver).
- A Fleet Owner can **assign any fleet truck to any affiliated driver**; that assignment **reflects in that driver's app**.
- A truck **does not need to be registered in the user's legal name** — we verify the **truck's authenticity (Vahan/RC)**, not ownership chain.
- The same truck can appear in **both** a Fleet Owner's account and an affiliated Driver's account.
- Roles are **fluid and migratable** — never a one-way, locked classification.

2.3 Three portals

1. **Shipper PWA** — load posting, auction/contract, tracking map, payment, history.
 2. **Driver/Fleet PWA → Android** — onboarding/KYC, truck & fleet management, load discovery, bidding/contracts, trip execution + GPS + checkpoints + POD, earnings.
 3. **Ops Console (web)** — KYC queue, live trip board, user/fleet/truck management, disputes/exceptions, manual overrides.
-

PART 3 — Current Build State (Reality Check)

Verified by reading every service's source. This is what we're building *from*, not what exists.

Module	Maturity	What works today	What's missing / broken for MVP
bt-auth-service	Partial	Custom JWT (email+pw, magic link, Google OAuth); onboarding routes	KYC fully stubbed (501) ; Surepass client throws "not implemented"; phone OTP only logs to console; register drops truck fields; not on Supabase Auth; ENCRYPTION_KEY missing from env example
bt-bookings-service	Partial	Direct + auction data model; bilateral counter-offer negotiation; blind-auction reads; Redis GPS ingest (30s TTL)	Lifecycle dead-ends at accepted (no pickup/delivery endpoints); no payment/pricing/ledger HTTP calls ; notifications are stubs; auctions never expire; award not transactional; no GPS persistence
bt-pricing-service	Functional	Rule-based static rate tables, distance × vehicle × load-mult, 10% fee	TS not Python ; RL/ML engine you described is NOT here ; no persistence; no quote IDs/locking; no auth; no demand/surge
bt-payment-service	Stub	Health check; route scaffolds with Zod shapes	Razorpay imported nowhere; order/escrow/capture/payout/refund all hardcoded stubs; no webhook signature check; no DB; no auth
bt-cargo-ledger	Stub	SHA-256/Merkle code; checkpoint route shapes	No DB persistence; no chain write (ethers not installed); booking never calls it; proof endpoint returns placeholder; no photo/EXIF storage
bt-tracking-service	Empty	Nothing (only node_modules)	Entire service unbuilt; no GCP Maps project/keys; no <LiveTrackMap> ; no breadcrumb table; plan docs lost from disk
bt-ops-web	Stub	Renders /ops and /portal screens; theme toggle	Fake login (no auth boundary) ; all data is mock; KYC approve/reject buttons inert; no live trips; no users/disputes screens; no tenant scoping
driver app	Partial + build broken	login, available loads, my-quotes, booking detail, profile	Onboarding wizard imports ~11 undefined fns → next build fails ; onboarding unreachable; GPS not field-ready (no wake-lock/store-forward); JWT in localStorage
shipper app	Partial + build broken	login, dashboard, new booking, negotiation UI	Imports missing getRoute / LiveTrackMap / @/lib/maps → build fails at booking detail ; untracked in git ; payment is trust-based "Mark as Paid"; lat/lng typed by hand
Platform/infra	Partial	GCP Cloud Run (asia-south1); keyless OIDC CI/CD; polyrepo per service	API gateway (bt-gateway) absent from repo → apps can't reach backend; no DB migrations in version control; empty env URLs; no tests; stale Render configs

Implication: ~30% built and most of the hard, money-and-trust-critical pieces (payment, ledger, KYC, tracking, ops) are at zero or stub. The schedule in Part 10 reflects this.

PART 4 — MVP Scope (In / Out)

4.1 Priority legend

- **P0** = MVP-critical; the "Completed Paid Trip" loop cannot happen without it.
- **P1** = strongly wanted; ship if time allows; degrade gracefully if not.
- **P2** = post-MVP.

4.2 Scope table

#	Capability	Priority	Owning module	Current state
1	Sign-up + login (Shipper, Driver/Fleet, Ops)	P0	auth, all apps	Partial (apps build-broken)
2	Truck-derived role model (driver↔fleet) + truck management	P0	auth, driver app	Not built
3	KYC: Driver PAN+Aadhaar, Truck Vahan/RC, insurance/permit/fitness collect+verify	P0	auth (Surepass)	Stub
4	Shipper KYC when order > ₹50,000	P0	auth	Stub
5	Manual KYC approval via Ops Console + Verified badge	P0	ops-web	Stub
6	Load posting (point-to-point, FTL+LTL, full field set)	P0	booking, shipper app	Partial
7	Auction booking (multi-driver bid, shipper picks, time-boxed)	P0	booking	Partial
8	Direct contract booking (1:1)	P0	booking	Partial
9	Bilateral negotiation, capped 5 rounds, deadline-expiry	P0	booking	Partial (expiry missing)
10	Cost-breakdown pricing (fuel/driver/per-km) as anchor	P0	pricing (CTO engine)	Rule-based exists; engine to port
11	RL dynamic-pricing layer (LinUCB) wired in (synthetic data)	P1	pricing (Python)	Not wired
12	Trip lifecycle: accept → pickup → in-transit → delivered → paid	P0	booking	Missing back half
13	Live GPS stream from driver app	P0	driver app, tracking	Empty
14	Shipper live map (Google Maps) + truck marker	P0	tracking, shipper app	Empty/broken
15	Geofence pickup/drop (denser GPS near zones)	P0	tracking	Not built
16	>1hr halt alert to shipper	P1	tracking	Not built
17	Deep-link to Google Maps for driver navigation	P0	driver app	Not built
18	Checkpoint capture: photo + EXIF GPS, stored in data sink	P0	cargo-ledger, driver app	Stub

#	Capability	Priority	Owning module	Current state
19	Blockchain hash-anchor per checkpoint/POD, ledger per order	P0	cargo-ledger	Stub (chain off)
20	POD = OTP to receiver email; OTP closes the trip	P0	booking, cargo-ledger	Not built
21	Payments: escrow (Razorpay) / direct / cash modes	P0	payment	Stub
22	Detention charge per hour	P1	payment, booking	Not built
23	Driver payout to bank	P0	payment	Stub
24	Ops Console: live trip board, full visibility, override	P0	ops-web	Stub
25	Notifications (≥ 1 channel: SMS/WhatsApp + push)	P0	booking, infra	Stub
26	Saved/secure per-user data (addresses, templates)	P1	apps	Not built
27	Fleet Owner reviews (no driver rating)	P1	booking/auth	Not built
28	API gateway + DB migrations in version control	P0	infra	Missing
29	Capacitor Android wrap of Driver app (native GPS)	P0*	driver app	Not built

* #29 is P0 for the real on-road pilot but P1 for the dev/test phase, which runs on the web PWA.

4.3 Explicitly OUT of MVP

Multi-pickup / multi-drop · partial/short delivery · mid-trip price renegotiation (booked price is final) · driver ratings · intracity loads · WhatsApp/voice booking · vernacular voice search · real-time automated driver-matching/dispatch · guided-pricing UI (deferred, data-dependent) · full RBI/PA escrow legal compliance · GST tax-invoice issuance · in-app turn-by-turn navigation (we deep-link).

PART 5 — Detailed Scope by Module

Each subsection: **Goal** → **Requirements** → **Current state** → **Gap to close** → **Acceptance criteria**.

5.1 Identity, Roles & KYC (*service: bt-auth-service + Ops Console*)

Goal. Every paying actor is a verified human/entity, every working truck is a verified vehicle, and roles flow from trucks.

Requirements.

- **Auth:** email+password, magic link, Google OAuth (already present). Phone OTP via a real gateway (MSG91) — *currently only logs to console*. Decide Supabase-Auth vs keep-custom (see Open Questions).
- **Truck-derived roles:** account starts role-less; attaching a verified truck makes it a Driver; a 2nd truck makes it a Fleet Owner; both can coexist; fully migratable.
- **Truck management:** add truck by RC number → **Vahan (RC) verification via Surepass**; verify the truck is genuine and roadworthy (fitness, insurance, permit expiry surfaced). Truck need **not** be in the user's name.
- **Fleet ↔ driver affiliation:** Fleet Owner can add affiliated drivers and assign a fleet truck to a driver; assignment reflects in the driver's app.
- **Driver KYC (minimum gate): PAN + Aadhaar** (Surepass aadhaar-v2 + pan), DL (HMTV/HTV/HGMV class enforced), bank account (penny-drop) for payouts.
- **Shipper KYC:** required **only when an order value exceeds ₹50,000** (PAN at minimum).
- **Approval: manual** — KYC documents land in the **Ops Console queue**; ops approves/rejects after Surepass + Vahan cross-check pass. Approved → **"Verified" badge** + trip-count shown.
- **Trust signals:** Verified badge + completed-trip count. **Fleet Owner reviews** allowed. **No driver ratings** at MVP.
- **PII:** AES-256-GCM for Aadhaar/PAN/bank numbers; SHA-256 hash for duplicate-identity detection (fraud check). Face-match image not stored.

Current state. Auth flows work; **the entire KYC layer is stubbed (501)**; Surepass client throws; phone OTP not wired; register drops truck fields; fraud checks throw; `ENCRYPTION_KEY` absent from env example.

Gap to close. Implement the 9 Surepass verification calls (at least PAN, Aadhaar, RC/Vahan, DL, bank, face-match); persist KYC records + 4-tier level gating (booking at L1, payouts at L3); wire MSG91 OTP; build truck CRUD + role derivation + fleet-driver affiliation; fix register to persist truck fields; ship the Ops KYC approval queue with real handlers.

Acceptance criteria.

- A new user can sign up, add a truck by RC, get it Vahan-verified, complete PAN+Aadhaar+bank KYC, and be **manually approved by ops** — after which the account shows "Verified" and can bid on loads.
- Adding a 2nd truck flips the account to Fleet Owner; assigning that truck to an affiliated driver appears in the driver's app.
- A shipper attempting a >₹50k order is blocked until shipper-KYC clears.

5.2 Load Posting & Booking (*service: bt-booking-service + Shipper app*)

Goal. A shipper posts a precise point-to-point load and gets it matched via auction or direct contract.

Requirements — the Load object (fields).

Field	Notes
Origin	Address + geocoded lat/lng (autocomplete, not hand-typed)
Destination	Address + geocoded lat/lng
Weight / Quantity	Tonnage or volumetric, by material type (feeds pricing)
Material type	Constrained list; drives pricing multipliers & quantity basis
Truck type	2-3 supported big-truck classes (see §5.4)
Pickup schedule	Date + time window
Auction end time	When bidding closes
Load mode	FTL or LTL
E-way bill	Optional (number if available)
Shipper details	Auto from profile
<i>(suggested additions)</i>	Material value (for >₹50k KYC trigger & insurance flag); special handling flags; contact-at-pickup

- **Scope limit: point-to-point only** (single pickup, single drop). No multi-leg.

Requirements — booking modes.

- **Auction:** shipper posts "into the void"; **multiple drivers/fleets bid**; **shipper chooses** the winner. Time-boxed by auction-end. Blind (a driver sees only their own quote).
- **Direct Contract:** shipper books a **specific** driver/fleet 1:1 (a known transporter). This is the "contract" path that must exist at MVP. *(Standing/recurring contracts — see Open Questions; MVP = single direct contract per load.)*
- **Booked price is final** — no mid-trip change except detention (§5.5).
- **Lifecycle:** `pending` → `negotiating` → `accepted` → `in_transit` → `completed` → `paid` (+ `cancelled`, `expired`). Cancellation only before `in_transit`.

Current state. Direct + auction data model, bilateral negotiation, and blind-auction reads exist. **Lifecycle dead-ends at `accepted`** — no pickup-confirm or delivery endpoints. Auctions never expire. Award is not transactional. No outbound calls to pricing/payment/ledger. Shipper app's booking-detail page **fails to build**.

Gap to close. Implement `accepted` → `in_transit` (pickup confirm) and `in_transit` → `completed` (delivery via POD-OTP) endpoints; implement auction expiry job; wrap award in a transaction; wire booking-create → pricing quote-lock and delivery → payment-release and checkpoint → ledger; geocoding/address autocomplete in the shipper form; fix the build.

Acceptance criteria.

- Shipper posts a point-to-point FTL load with all fields; it appears to eligible drivers.
- Auction: ≥2 drivers bid; shipper picks one; losers auto-expire at deadline.
- Direct: shipper sends a contract to one fleet; fleet accepts; booking moves to `accepted`.
- The trip can progress all the way to `paid`.

5.3 Negotiation (*service: bt-booking-service*)

Goal. Transparent price discovery between shipper and driver — a *feature*, not friction.

Requirements.

- **Bilateral counter-offers** — both shipper and driver can counter.
- **Capped at 5 counter-offers**, then no more.
- **Time-limited** by the auction-end / offer-expiry.
- Append-only **negotiation history** (immutable audit log, one row per offer) — already modeled.
- In auctions, **multiple drivers** negotiate in parallel; **shipper selects** the winner.

Current state. Bilateral counter (PATCH counter) + append-only `negotiations` table exist. No round-cap enforcement; no auction expiry.

Gap to close. Enforce the 5-round cap; enforce deadline expiry; surface the cap/clock in both apps.

Acceptance criteria. A negotiation thread stops accepting counters after 5 rounds or at deadline, whichever first; the full thread is visible to both parties.

5.4 Pricing Engine (*service: bt-pricing-service — to become Python/FastAPI; see Appendix A*)

Goal. Given load parameters, return a **price with a transparent breakdown** (fuel, driver wage, per-km distance, handling) — and, where data permits, a **demand-aware dynamic quote**.

Requirements.

- **Truck classes:** SCV / LCV / MCV / HCV — MVP supports **2-3 big-truck classes** (MCV/HCV range) that actually run Indian highways.
- **Cost-breakdown engine (the anchor, P0):** the **CTO engine** — input `(truck_category, distance, age)` → deterministic operating-cost breakdown using market constants (mileage by BS4/BS6/BS6-Ph2 norm, AdBlue/DEF %, service cost by age, engine+gear oil cost, diesel price, driver wage, fixed cost, capacity, handling extra). **Quantity basis varies by material** — tonnage for some, volumetric for others.
- **Dynamic layer (P1, data-dependent):** the **LinUCB contextual-bandit RL agent** quotes against a 16-feature market context (demand/supply, monsoon, harvest season, etc.). At MVP this trains on **synthetic data** (`pretrain.py` / `market_sim.py`); real accuracy needs the data in Part 7. A second agent (`LinearQAgent`, backhaul-aware) exists but is **not used** — keep out of MVP.
- **Quote integrity:** persist a **quote ID** so the price shown is the price locked at booking.

Current state. The **deployed** service is a TS rule-based static rate-card (works, but is the placeholder). The **RL engine you described (CTO data/engine + LinUCB + market sim + pretrain) is a separate Python codebase not yet integrated**. No quote persistence, no auth.

Gap to close. Stand up the Python/FastAPI pricing service; port the CTO cost-engine to produce the breakdown; expose `/quote`; persist quote IDs; integrate booking → quote-lock. Wire LinUCB on synthetic data behind a flag.

Acceptance criteria. Posting a load returns a price with a line-item breakdown (fuel / driver / per-km / handling); the quoted price is the price charged at booking.

Pricing is the most data-hungry part of the product — its real-world correctness depends on the procurement list in **Part 7**. Treat the MVP pricing as "directionally right with synthetic/seed data," not "market-accurate."

5.5 Payments & Escrow (*service: bt-payment-service*)

Goal. Move money for a real trip — flexibly — while the legal wrapper is still being decided.

Requirements.

- **Multiple modes: Escrow (Razorpay), Direct** (shipper→driver via platform), and **Cash** (recorded off-platform, marked in-app).
- **Escrow flow:** shipper funds on booking → held → released to driver's verified bank on **delivery confirmation (POD-OTP)**.
- **Payout:** to driver/fleet bank account (penny-drop verified at KYC).
- **Detention:** configurable **charge per hour** when pickup/drop is delayed beyond the window.
- **Refund:** on cancellation/no-show (ops-triggered at MVP).
- **Provider:** Razorpay (+ RazorpayX for payouts).
- **Compliance:** *MVP explicitly does **not** need to pass full payment/escrow legal compliance — this is a feasibility test.* Razorpay + "follow legal things" is the **intended** path but **TBD**; flag every money endpoint as compliance-pending.

Current state. Stub. Razorpay package present, imported nowhere; all endpoints hardcoded; no webhook signature verification; no DB; no auth on `/release`.

Gap to close. Real Razorpay order-create + webhook (HMAC-verified) + capture + RazorpayX payout + refund; an escrow state machine (held/released/refunded per booking); cash-mode recording; detention calc; idempotency + auth on money endpoints.

Acceptance criteria. For at least the **escrow** mode: shipper funds a booking, money is held, and on POD-OTP the driver receives a payout to their bank — observable end-to-end on a real (test-mode acceptable) transaction. Cash and direct modes recorded correctly.

⚠ **Risk flag:** holding shipper funds and releasing to drivers makes us a payment intermediary; doing this with real money at any scale invokes RBI PA/escrow rules. For the MVP feasibility test, prefer **Razorpay Route / escrow-as-a-service** (Razorpay holds custody) over self-custody, and keep volumes tiny. *(Compliance is a TBD per your direction — see Open Questions & Risks.)*

5.6 Live Tracking & Maps (*new service: bt-tracking-service + both apps*)

Goal. The shipper sees their truck move on a live map; the driver navigates without us building a navigator.

Requirements.

- **Driver app streams GPS** on a regular cadence; location is held **in-app/in-platform** (not just on the device).
- **Cadence (recommended):**
 - **Geofenced zones (near pickup & drop, e.g. 2-3 km radius):** dense — every **~10-15 s**.
 - **Mid-trip / highway:** every **~60 s** (trips run 20+ hrs — balance battery/data).
- **Transport:** for fast refresh and many concurrent trucks, ingest via a stream (**Kafka** is your stated preference; Redis-pub/sub is the lighter MVP alternative — see Open Questions). Persist a throttled **breadcrumb trail** (~1 point/10-15 s) for audit/dispute.
- **Shipper map (P0):** Google Maps with a moving truck marker, route polyline, and ETA.
- **Halt alert (P1):** if the truck is stationary **> 1 hour**, surface it on the shipper side.
- **Geofencing (P0):** auto-detect arrival at pickup/drop; trigger denser sampling + status hints.

- **Navigation: deep-link to the phone's Google Maps app — no in-app turn-by-turn.**
- **Provider: Google Maps Platform — Maps JS + Routes API (New) + Places API (New)** (legacy Directions/Places are blocked for new GCP projects). Two restricted keys (browser referrer-restricted + server secret) + per-API quota caps for cost control.
- **Web limitation & plan:** browsers can't do reliable background GPS (esp. iOS). MVP dev/test runs on the **web PWA** (screen-on / wake-lock); the **real pilot runs the Capacitor-wrapped Android Driver app** for native background GPS.

Current state. Empty service. No code, no GCP Maps project/keys, no `<LiveTrackMap>` component; both apps reference map modules that don't exist (build-breaking). Existing raw GPS ingest lives in booking-service (Redis, 30s TTL, no persistence).

Gap to close. Phase-0 GCP Maps setup (3 APIs + 2 keys + quota caps) **before any map code**; build bt-tracking-service (route/ETA proxy with caching, geofence logic, halt detection, breadcrumb persistence); choose & wire the ingest transport (Kafka vs Redis); build `<LiveTrackMap>` in both apps; add wake-lock + store-and-forward to the driver app; build the deep-link nav helper.

Acceptance criteria. On a real (or simulated) drive, the shipper watches the truck move on a map with a live position updating at the cadences above; geofence entry at pickup/drop is detected; a >1hr halt is flagged.

5.7 Proof of Delivery, Checkpoints & Cargo Ledger (*service: bt-cargo-ledger + driver app*)

Goal. Make delivery and chain-of-custody **tamper-evident and trip-closing** — this is the legal/trust backbone, and it stays in MVP.

Requirements.

- **POD = receiver-email OTP:** at delivery, an **OTP is sent to the receiver's email**. The **receiver can be a different person from the shipper and needs no account**. Entering the correct OTP is the **key that completes the trip** (drives `in_transit → completed` and triggers payout).
- **Checkpoints:** at each checkpoint along the trip, the driver captures **photos** that carry **EXIF metadata incl. GPS** (location of capture). Photos are stored by us in a **data sink** (object storage — R2 / Supabase Storage).
- **Ledger / blockchain (core, kept):** when a checkpoint/POD artifact is captured, we compute its **hash** and **write a blockchain transaction** anchoring that hash. All a trip's checkpoint+POD hashes form a **ledger clubbed to the order** — this **chained record is the legal proof** of the shipment's chain-of-custody and delivery.
- **Design:** SHA-256 canonical-JSON leaf hashing → Merkle tree per shipment → anchor the root (and/or per-checkpoint hashes) on a low-cost chain (Polygon was the design choice). Typed checkpoints (pickup/handoff/waypoint/delivery).

Current state. Stub. Merkle/SHA-256 code exists but **no DB persistence, no chain write (ethers.js not installed), no photo/EXIF storage**, and **booking never calls the ledger**. Blockchain gated off (`BLOCKCHAIN_ENABLED=false`).

Gap to close. Persist checkpoints+shipments (Supabase); implement photo upload with EXIF/GPS extraction → data sink; implement the OTP-to-receiver flow + verification that closes the trip; install ethers.js and implement the chain write (decide chain/wallet); build the `/proof` endpoint that recomputes & verifies the chain; wire booking checkpoints/delivery → ledger.

Acceptance criteria. A delivery is completed only when the receiver enters the emailed OTP; each checkpoint photo is stored with its GPS EXIF; each checkpoint/POD has an on-chain hash; the per-order proof endpoint

returns a verifiable chain-of-custody.

Note: the chain anchor is a **trust feature**, not a legal substitute for contracts/insurance. It proves "these artifacts existed at these times unaltered," which is valuable, but treat its "legal" status as supporting evidence (and confirm with counsel later).

5.8 Ops Console (*service: bt-ops-web*)

Goal. One operator, one screen, total visibility; touch the flow only on exceptions.

Requirements (P0 unless noted).

- **Real auth + RBAC** (ops staff only) — *currently fake login*.
- **KYC approval queue:** see pending Driver/Truck/Shipper KYC with Surepass+Vahan results; **approve/reject** (real handlers); issue Verified badge.
- **Live trip board:** every in-progress trip across all drivers/fleets with live position, status, ETA, halts.
- **Users/Fleets/Trucks management:** search, view, suspend; see fleet→driver→truck affiliations.
- **Exceptions / disputes:** view a trip's full state (negotiation log, GPS trail, checkpoints, ledger, payment) and **manually override** — reassign on driver no-show/breakdown, trigger refund, force payout, edit a stuck booking.
- **(P1)** audit trail of who-approved/overrode-what; notifications inbox.

Current state. Stub. Fake login (no boundary), all mock data, KYC buttons inert, no live trips, no users/disputes screens, no tenant scoping. Serves /ops and /portal from one codebase.

Gap to close. Real auth+RBAC; replace all mock data with BFF calls to the services; implement KYC queue handlers; live trip board fed by tracking; user/fleet/truck management; an exception view with override actions.

Acceptance criteria. Ops can approve a real KYC, watch a real trip live, and manually intervene (reassign / refund / force-complete) on a stuck trip — all against real platform data.

5.9 Notifications (*booking-service + infra*)

Goal. Tell users when something needs them — at least one reliable channel.

Requirements. Notify on: new load (to eligible drivers), new quote/counter (to the other party), award/loss, status changes (picked up / in transit / delivered), payment events, KYC result. **At least one channel at MVP** — recommend **SMS (MSG91) and/or WhatsApp** for drivers (works on cheap phones, no app open) + **web push** for shippers. (*Channel mix is an Open Question.*)

Current state. All notification jobs are stubs (MSG91/Firebase env unused).

Gap to close. Wire one real channel end-to-end; queue-based fan-out.

Acceptance criteria. A driver is notified of a new matching load and an award without having the app open.

5.10 Platform / Infrastructure (*cross-cutting*)

Goal. A reproducible, reachable stack the pilot can actually run on.

Requirements (P0).

- **API gateway (bt-gateway):** the Nginx reverse proxy that maps app `/api/*` → service routes — **must be in the repo, deployed, and configured** (apps currently can't reach the backend without it).
- **DB migrations in version control:** Supabase schema as committed migrations (currently only comments in code) so the pilot DB is reproducible.
- **Real env config:** production `NEXT_PUBLIC_API_URL` and per-service URLs (currently empty → default to localhost).
- **Fix builds:** both apps must `next build` green (they don't today).
- **Driver app → Capacitor Android:** wrap the Next.js Driver app in Capacitor for native background GPS before the on-road pilot.
- **(P1)** smoke tests on the money/booking paths; Cloud Run min-instances to avoid cold-start latency on OTP/booking; basic error tracking (Sentry) + log routing.

Current state. GCP Cloud Run + keyless OIDC CI works per service; **gateway absent from repo**; no migrations in git; empty env; stale Render configs; no tests.

Acceptance criteria. From a clean environment, the stack can be brought up, the apps reach the backend through the gateway, the DB is provisioned from migrations, and both apps build and deploy.

PART 6 — The Three Core User Journeys

6.1 Shipper journey

Post load (point-to-point, FTL/LTL) → receive auction bids **or** send a direct contract → negotiate (≤5 rounds, deadline) → **pick winner** → fund (escrow/direct/cash) → **watch truck live on map** → get halt/checkpoint updates → receiver enters delivery OTP → trip closes → settlement. *Breaks today at:* booking-detail page (build), tracking (empty), payment (stub).

6.2 Driver / Fleet Owner journey

Sign up → add truck (Vahan/RC) → KYC (PAN+Aadhaar+bank) → **ops approves** → **Verified** → (fleet) add trucks + affiliate drivers + assign → browse loads → **bid (auction) or accept contract** → negotiate → win → deep-link navigate → **stream GPS** → capture checkpoint photos (EXIF) → at drop, receiver OTP closes trip → **get paid** to bank. *Breaks today at:* onboarding (build-broken & unreachable), KYC (stub), tracking/checkpoints/POD (empty/stub), payout (stub).

6.3 Ops journey

Review KYC queue (Surepass+Vahan) → approve/reject → monitor all live trips → on exception (no-show, breakdown, dispute, stuck payment) → open trip detail (negotiation log + GPS trail + checkpoints + ledger + payment) → **override** (reassign / refund / force-complete). *Breaks today at:* everything (fake login, all mock).

PART 7 — Data Requirements (Standalone — for procurement)

Per your instruction: features that need data are isolated here so the data can be **requested/sourced separately**. Each line = a dataset + why + an MVP fallback.

7.1 Pricing engine data (highest priority — accuracy depends on it)

Data needed	Used by	MVP fallback	Source to pursue
Vehicle parc — real count of trucks per category/age on target lanes	market sim (demand/supply realism)	Synthetic counts in <code>cto_data.py</code>	Industry/RTO/SIAM data; later our own DB
Real demand–supply signal per corridor/time	LinUCB context (16 features)	Synthetic generator	Our own marketplace data once live; seed from partner fleet
Weather / monsoon API	market sim (monsoon feature)	Random variable	IMD / a weather API
Harvest-season calendar by region/crop	market sim (harvest feature)	Random variable	Agri calendars / govt data
Live diesel price by state	CTO fuel cost	Hardcoded ₹/L (~90)	Fuel price API / OMC feeds
Truck cost constants — mileage by BS norm, AdBlue/DEF %, service cost by age, oil costs, driver wage, fixed cost, capacity, handling premiums	CTO engine	Hardcoded in <code>cto_data.py</code>	Validate with the partner fleet's real numbers
Real road distance & tolls per lane	price + ETA	Caller-supplied distance	Google Routes API (already planned)

7.2 Maps & tracking data

Data needed	Used by	Notes
GCP Maps Platform project + 3 APIs (Maps JS, Routes New, Places New) + 2 restricted keys + Map ID	tracking, both apps	Phase-0 gating task — nothing renders without it
Per-API quota caps + billing budget	cost control	Hard cap = quotas; budget only alerts

7.3 KYC / verification data

Data needed	Used by	Notes
Surepass account + API key (Aadhaar v2, PAN, RC/Vahan, DL, GST, bank penny-drop, face-match)	auth KYC	Currently key read but client throws
MSG91 account (SMS/OTP) + WhatsApp provider	auth OTP, notifications	OTP only logs today

7.4 Payments data

Data needed	Used by	Notes
Razorpay merchant account (+ RazorpayX, + Route/escrow if available) keys & webhook secret	payment	Keys blank today
Registered entity for merchant onboarding	t	See Risks/Open Questions

7.5 Ledger / storage data

Data needed	Used by	Notes
Object storage bucket (R2 / Supabase Storage) for checkpoint photos	cargo-ledger	The "data sink"
Blockchain endpoint + wallet (Polygon or chosen chain)	cargo-ledger	ethers.js + funded wallet for anchoring

PART 8 — Non-Functional Requirements

- **Security:** real auth boundary on Ops Console (currently none); auth on money endpoints; webhook signature verification; PII encryption (AES-256-GCM) + hashed lookups; move JWTs toward httpOnly cookies (currently localStorage = XSS-exposed); secrets via Cloud Run secret manager, never in env examples.
 - **Reliability:** the booking + payment + POD path must be transactional/idempotent (award currently non-atomic; money endpoints non-idempotent). Cloud Run min-instances to avoid cold-start latency on OTP/booking.
 - **Data integrity:** DB migrations in version control; GPS breadcrumbs persisted for dispute resolution.
 - **Performance:** tracking ingest sized for N concurrent trucks at the stated cadences; map proxy responses cached.
 - **Observability:** error tracking (Sentry) + structured logs + uptime on the core path; an audit trail of ops overrides.
 - **Accessibility / UX:** Driver app must work for **non-literate users** — icon-led, minimal text, vernacular-ready; large tap targets; works on cheap Android, intermittent data, screen-on for GPS.
 - **Compliance (flagged, deferred):** payment-intermediary (RBI PA/escrow), KYC/PII handling (DPDP Act), goods-liability terms. **MVP intentionally does not clear these** — feasibility test only. Track as launch-blockers for the *real* (post-feasibility) launch.
-

PART 9 — Out of Scope / Deferred (with rationale)

Item	Decision	Why
AI dynamic pricing as primary	Defer to P1 , ship cost-breakdown anchor first	RL needs real data (Part 7); synthetic-trained at MVP
LinearQAgent (backhaul)	Out	Not wired anywhere; adds complexity
Multi-pickup / multi-drop	Out	Point-to-point only at MVP
Partial / short delivery	Out	Simplifies POD/ledger
Mid-trip price changes	Out	Booked price is final (detention is the only add)
Driver ratings	Out	Fleet-owner reviews only
Intracity loads	Out	Interstate + intrastate only
WhatsApp / voice booking, vernacular voice	Out	Post-MVP differentiators
Real-time automated dispatch	Out	Shipper picks manually
In-app turn-by-turn nav	Out	Deep-link to Google Maps
Full payment/legal compliance	Deferred	Feasibility test per founder direction
GST tax-invoice issuance	Out	Not needed for feasibility

PART 10 — Release Plan & 8-Week Timeline (30 Jun → 31 Aug 2026)

Aggressive given ~30% built. Sequenced so a thin **happy-path slice is demoable early**, then hardened. Treat as the plan to defend/cut against, not a guarantee.

Week	Theme	Key deliverables
W1	Foundations & unblock	Fix both app builds; commit shipper app to git; API gateway in repo + deployed ; DB migrations in version control; real env URLs; GCP Maps Phase-0 (APIs+keys+caps)
W2	Identity & roles	MSG91 OTP; truck-derived role model + truck CRUD + Vahan/RC verify ; Driver PAN+Aadhaar+bank KYC (Surepass); fleet↔driver affiliation
W3	Ops Console core	Real auth+RBAC; KYC approval queue (real handlers) + Verified badge; users/fleets/trucks views
W4	Booking loop closure	Pickup/delivery lifecycle endpoints; auction expiry; 5-round negotiation cap; pricing quote-lock ; shipper geocoding/autocomplete
W5	Pricing engine	Python/FastAPI pricing service; CTO cost-breakdown live; LinUCB on synthetic data (flagged); booking integration
W6	Tracking & maps	bt-tracking-service; GPS ingest (Kafka/Redis decision) + breadcrumbs; shipper live map ; geofencing; deep-link nav; driver wake-lock/store-forward
W7	POD, ledger & payments	Receiver OTP-closes-trip ; checkpoint photo+EXIF → data sink; chain anchor + per-order proof ; Razorpay escrow + payout + cash/direct modes ; detention
W8	Harden, wrap & pilot-prep	Capacitor Android wrap of Driver app; ops override actions; ≥1 notification channel; end-to-end happy-path test on the corridor; halt alerts; bug-bash

Rollout stages: internal alpha (us, simulated GPS) → **closed pilot on 1 corridor** with the partner fleet → broaden to more lanes/users. (Geography is any PAN-India corridor; pick the partner fleet's busiest lane first.)

Honest scope warning: W5-W7 each contain a from-scratch, trust-critical subsystem (pricing RL, tracking, ledger+payments). If the timeline slips, the safe cut order is: RL dynamic layer → halt alerts → detention → reviews → escrow-down-to-direct/cash-only — **never** cut the lifecycle closure, tracking map, POD-OTP, or KYC gate, because those define "Completed Paid Trip."

PART 11 — MVP Definition of Done / Deliverables Checklist

The MVP is "done" when an external user completes this loop unaided:

- ☐ A real Fleet Owner signs up, adds a Vahan-verified truck, passes PAN+Aadhaar+bank KYC, and is **ops-approved** → **Verified**.
- ☐ A real Shipper posts a **point-to-point interstate FTL/LTL** load with the full field set.
- ☐ Drivers **bid in an auction** (or the shipper sends a **direct contract**); negotiation respects the 5-round cap + deadline; **shipper picks** the winner.
- ☐ The shipper sees a **price with a fuel/driver/per-km breakdown**, locked at booking.
- ☐ The shipper **funds** the trip (escrow, or direct/cash recorded).
- ☐ The driver runs the trip; the shipper **watches the truck live on a map**; geofence + (≥)1hr-halt signals work.
- ☐ The driver captures **checkpoint photos with GPS EXIF**, stored in our sink, each **hash-anchored on-chain**, clubbed into a **per-order ledger** with a verifiable proof.
- ☐ At drop, the **receiver enters the emailed OTP** → trip **completes**.
- ☐ The driver/fleet **receives payout** to their bank (escrow mode) or settlement is recorded (cash/direct).
- ☐ **Ops** can watch every trip live and **manually override** a stuck/exception trip from the console.
- ☐ The **Driver app runs as a Capacitor Android app** with native background GPS.
- ☐ At least **one notification channel** delivers load/quote/status/payment alerts.

Engineering hand-over deliverables:

- ☐ All services build, deploy on Cloud Run, and are reachable through the gateway.
 - ☐ DB schema reproducible from committed migrations.
 - ☐ Both PWAs build green; Driver Android APK produced.
 - ☐ Env/secret config documented; Surepass/MSG91/Razorpay/Maps/storage/chain keys wired.
 - ☐ This PRD + an updated per-service README reflecting reality.
-

PART 12 — Risks & Mitigations

Risk	Severity	Mitigation
Scope vs 8-week deadline — 5 trust-critical subsystems are at zero/stub	High	Ruthless happy-path focus; the W10 cut-order above; consider extra hands on tracking+payments
Payment-intermediary legality (RBI PA/escrow)	High	Prefer Razorpay Route/escrow-as-a-service over self-custody; tiny volumes; counsel before real-money scale (deferred but flagged)
KYC/PII handling (DPDP) at real scale	Med	Encrypt + minimize; consent copy; deferred for feasibility but a launch-blocker later
Web background GPS unreliable	High (for tracking)	Web only for dev/test; Capacitor Android for the on-road pilot; wake-lock + store-forward
Pricing accuracy on synthetic data	Med	Ship deterministic cost-breakdown as the anchor; treat RL as directional; procure Part-7 data
Blockchain anchoring cost/complexity	Med	Anchor one Merkle root per shipment (not per checkpoint) on a low-cost chain; keep off-chain hashing primary
Cold-start latency on Cloud Run free tier hurting OTP/booking	Med	min-instances on hot services
Single ops person as the safety net	Med	Make override actions first-class in the console; on-call playbook for no-show/breakdown
Goods loss/damage liability undefined	Med	Clear terms-of-use disclaimer at MVP; insurance partnership later
Non-literate driver UX	Med	Icon-led flows; usability test with a real driver early (W3-W4)

PART 13 — Open Questions / TBD Decisions (need your call)

1. **Payments mode for the *demo*:** which mode must work end-to-end first — **escrow**, or is **direct/cash recorded** enough to prove feasibility? (Affects W7 load.)
 2. **Razorpay custody:** Razorpay **Route/escrow-as-a-service** (they hold funds) vs platform self-custody? (Legal exposure differs hugely.)
 3. **Registered entity** ready for Razorpay/Surepass merchant onboarding? (Blocks real keys.)
 4. **GPS ingest transport:** **Kafka** (your preference, heavier to operate) vs **Redis pub/sub** (lighter, faster to ship for a pilot)? Recommend Redis for MVP, Kafka post-pilot.
 5. **Auth:** finish the **Supabase Auth** migration or keep the working **custom JWT** for MVP? Recommend keep custom JWT (it works) and defer Supabase.
 6. **Notification channel:** SMS (MSG91) vs WhatsApp vs both, for the one MVP channel? Recommend WhatsApp + push.
 7. **"Contract" semantics:** is the MVP contract just a **single direct 1:1 booking**, or do you also need **standing/recurring contracts** (repeat lanes)? Recommend single direct for MVP; recurring post-MVP.
 8. **Blockchain choice:** Polygon (as designed) vs another chain, and who funds the anchoring wallet?
 9. **Supported truck classes:** confirm the exact **2-3 truck types** (e.g. specific MCV/HCV configs) for pricing + load forms.
 10. **Native app build:** you asked if Android dev is feasible solo — **yes, via Capacitor wrapping the existing Next.js Driver app** (~95% reuse, native GPS). Confirm we go this route (vs staying web-only for the very first tests).
-

Appendix A — Pricing Engine Spec (from founder's notes)

A separate **Python** RL pricing system (the original "AI/Pricing microservice"), to be exposed via FastAPI and integrated with booking.

- **cto_data.py** — market knowledge & constants. Truck categories **SCV** (Small), **LCV** (Light), **MCV** (Medium), **HCV** (Heavy). Mileage in km/yr + kmpl by emission norm (BS6 / BS4 / BS6-Ph2); **DEF6** = % of diesel consumption used as AdBlue (emission fluid). **vehicle_parcs** = trucks per category (synthetic now → DB later) to simulate demand/supply; age drives emission law & fluid cost. **service_cost** = maintenance by truck type & age (rough rule: older → more repairs). **oil_cost** = engine-oil & gear-oil cost per change (total oil + km by BS norm). **Defaults** = diesel cost, driver wage, fixed cost, per-truck capacity, object-handling extra cost. *All hardcoded today → automate/DB later.*
- **cto_engine.py** — `CT0Engine.breakdown(category, distance, age)` → deterministic operating-cost **breakdown** (fuel / driver / per-km / handling). **This is the MVP price anchor.**
- **pretrain.py** — generates random data to train the RL agent and saves weights for reuse.
- **market_sim.py** — market env with **16 features** the agent observes. **Monsoon** (random now → real **weather API**), **harvest season** (random now → real **harvest calendar**), **demand/supply** (synthetic now → real data). *All three need real data to be correct (Part 7).*
- **rl_agent.py** — two agents: **LinUCB** (contextual bandit; one ridge-regression model per action) = the dynamic-quote engine; **LinearQAgent** (reward includes future context incl. backhaul) = **not currently used** → **out of MVP**.

MVP stance: ship the **CTO deterministic breakdown** as the anchor (P0); wire **LinUCB on synthetic/seed data** behind a flag (P1); procure Part-7 data to make it market-accurate post-MVP.

Appendix B — Tech Stack & Conventions (as built)

- **Backend services:** Node 20 + TypeScript + **Fastify** + Zod; **pricing** to be **Python + FastAPI**.
 - **DB:** Supabase **Postgres** (service-role; lat/lng as decimals, no PostGIS). **Redis** (Upstash/ioredis) for cache/OTP/sessions/live location.
 - **Frontends:** **Next.js 16 / React 19 / Tailwind 4** PWAs (Shipper, Driver/Fleet, Ops). **Capacitor** for Android.
 - **Maps:** Google Maps Platform — Maps JS + Routes API (New) + Places API (New); deep-link nav.
 - **Payments:** Razorpay (+ RazorpayX; consider Route/escrow). **KYC:** Surepass (incl. Vahan). **OTP/SMS:** MSG91. **Notifications:** + WhatsApp / FCM push.
 - **Storage:** object store (R2 / Supabase Storage) for checkpoint photos. **Chain:** Polygon (or TBD) for ledger anchoring via ethers.js.
 - **Infra:** **GCP Cloud Run** (asia-south1); **Nginx gateway (bt-gateway)** as single edge `/api/*` → services; **keyless OIDC** CI/CD; polyrepo per service.
 - **Conventions:** snake_case JSON; `{success, data, code, message}` envelope; client JWT access+refresh (migrate toward httpOnly cookies).
-

End of PRD v1.0 — draft for review. Update the build-state table and timeline as work lands; treat Part 7 (Data) and Part 13 (Open Questions) as live procurement/decision trackers.